

组原第五次实验报告

实验名称：单-多周期CPU

学号：2312966 姓名：林晖鹏 班次：张金老师

目录

1. 实验目的
 2. 实验内容
 3. 单周期原理图
 4. 单周期原理解释
 5. 单周期bug修复
 6. 单周期验证仿真功能
 7. 单周期对三种mips指令每个挑一个进行分析
 8. 多周期CPU原理解释
 9. 多周期CPU bug修复
 10. 多周期CPU添加功能
 11. 多周期CPU验证仿真功能
-

一、实验目的

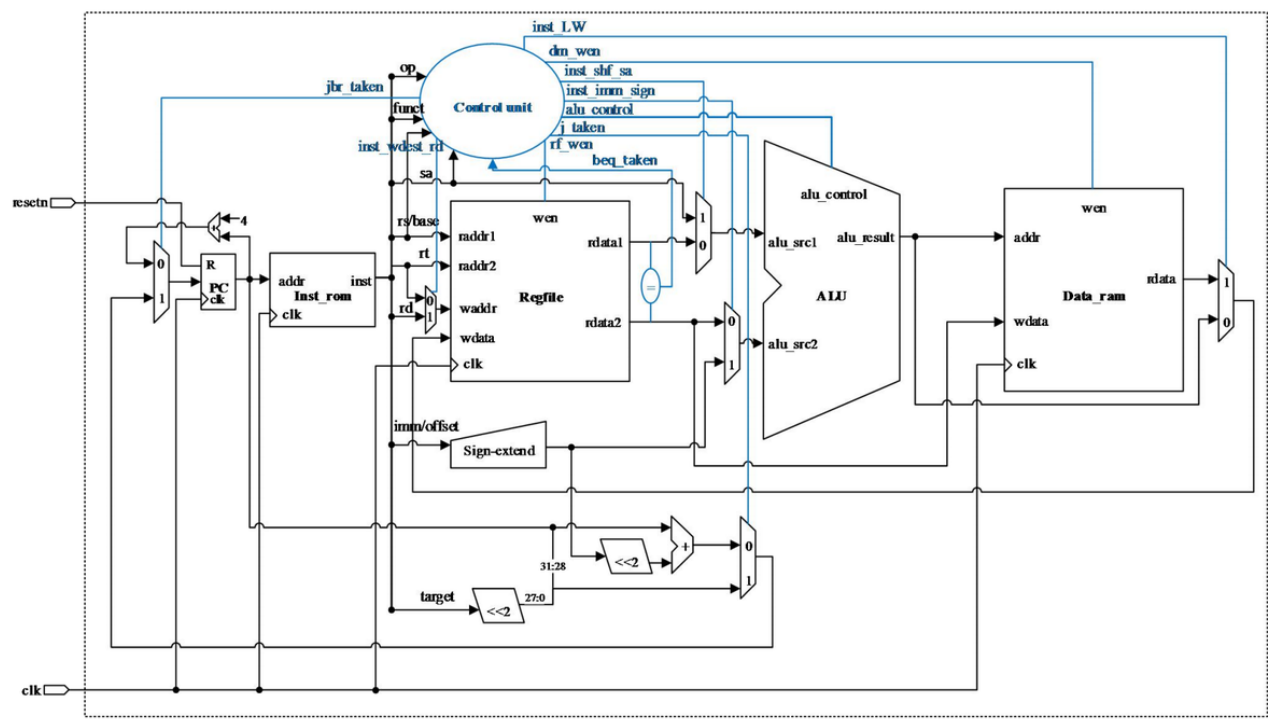
- 理解 MIPS 指令结构，理解 MIPS 指令集中常用指令的功能和编码，学会对这些指令进行归纳分类。
 - 了解熟悉 MIPS 体系的处理器结构，如延迟槽，哈佛结构的概念。
 - 熟悉并掌握单周期 CPU 的原理和设计。
 - 进一步加强运用 verilog 语言进行电路设计的能力。
 - 为后续设计多周期 cpu 的实验打下基础。
-

二、实验内容

- 1. 复现单周期cpu，并且剖析MIPS指令，同时对三种类型的MIPS指令，挑1~2条具体分析总结其执行过程
- 2. 然后根据单周期cpu找出并修改多周期cpu中的bug，实现多周期cpu。
- 3. 将之前ALU实验中添加的三种功能添加到单周期和多周期cpu里面。
- 4. 仿真或者上实验箱进行验证。

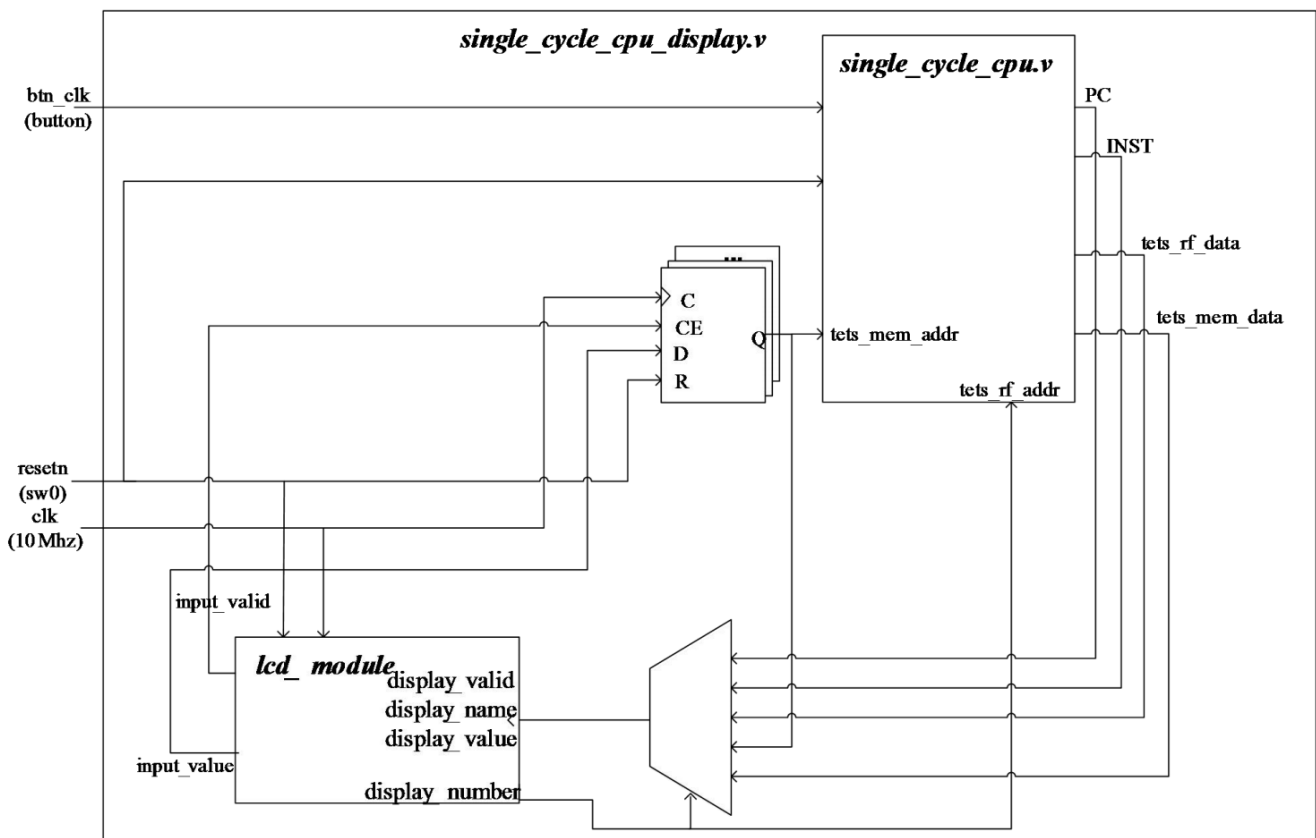
三、实验原理图

1. 单周期CPU原理图



在

2. 单周期CPU顶层模块图



四、单周期CPU原理解释

CPU的执行流程一般分为下面五个部分

- 取指：从指令存储器中读取当前 PC 所指向的指令。
- 译码：解析指令各个字段，如操作码、寄存器编号、立即数等。
- 执行：执行对应的算术逻辑运算或地址计算
- 访存：如是访存指令（如 `LW`，`SW`），则访问数据存储器。
- 写回：将结果写回寄存器堆。

单周期CPU指的是一种将一条指令的完整执行流程在一个时钟周期内完成的处理器架构。在一个周期中执行完上面的所有流程

五、单周期bug修复

一开始直接仿真验证的时候，发现全都是XXXX。检查代码发现在下面这个位置不符合逻辑。

与运算左边的是一个四位二进制，而右边的为一位二进制。在进行与运算的时候就只会最低位进行运算，得到的结果并不是理想的结果。

所以这里我们把 `resetn` 变量扩展为四位，再进行与运算。

代码块

```
1 //修改前
2 assign dm_wen    = {4{inst_SW}} & resetn;    // 内存写使能,非resetn状态下有效
3 // 修改后:
4 assign dm_wen    = {4{inst_SW}} & {4{resetn}};    // 内存写使能,非resetn状态下有效
```

六、单周期仿真验证

为了方便验证功能正确性，我们设计仿真文件：

我们用阻塞赋值以及加入#1间隔，用rf_addr这个接口，来读取寄存器堆里面的值，从而验证计算结果是否正确。

这里加入阻塞赋值是防止下一个读取地址的结果覆盖上一个的结果。

使用#1间隔是防止操作没有完成，以及上一次读取寄存器堆操作没有完成。

代码块

```
1 // 连接到CPU内部信号
2 assign op = last_pc[31:26];
3 assign rs = last_pc[25:21];
4 assign rt = last_pc[20:16];
5 assign rd = last_pc[15:11];
6 assign sa = last_pc[10:6];
7 assign funct = last_pc[5:0];
8 assign imm = last_pc[15:0];
9
10 always @(posedge clk) begin
11     last_pc <= cpu_inst;
12
13     rf_addr <= rs;
14     #1
15     rs_data <= rf_data;
16     rf_addr <= rt;
17     #1
18     rt_data <= rf_data;
19     rf_addr <= rd;
20     #1
21     rd_data <= rf_data;
22     rf_addr <= sa;
23     #1
```

```

24         sa_data <= rf_data;
25
26     end

```

仿真结果如下图：

可以看到，比如第二个语句，汇编指令为 `sll $2, $1, #4`，寄存器\$2上的结果应该是 `0000_0010H`，从图中我们可以看到 `rd_data` 的值恰好为 `0000_0010`，所以结果是正确的。

其余正确性我就不逐个分析了。



七、单周期MIPC指令分析

接下来，我们分别对三类指令类型的某条指令进行分析。

（一）R型指令

在 MIPS 指令集中，R 型指令用于三寄存器之间的操作，例如加法、减法、逻辑运算、移位等。这类指令通常不涉及内存，只操作寄存器中的值。

1. R型指令格式

代码块

```

1  31      26 25      21 20      16 15      11 10      6 5      0
2  |  op      |  rs      |  rt      |  rd      |  sa      |  funct  |

```

- `op`：恒为000000；

- rs、rt：两个源操作数所在的寄存器号；
- rd：目的操作数所在的寄存器号；
- sa：位移量，决定移位指令的移位位数；
- func：功能位，决定R型指令的具体功能。

2. 案例分析

addu是无符号数加法，这里把寄存器\$1、\$2存储的数据相加之后存到寄存器\$3中。

2.1 取指

首先是从指令集中取值，我们取出来指令为 00000000 01100100 00101000 00100011。

2.2 译码

然后根据对指令译码，我们拆解出op、rs、rt、rd、sa、func这些字段。

并处理为对应信号，以及访问rs、rt对应寄存器的数据，得到操作数。

这里我们得到\$1的数据为0000 0001H；\$2的数据为0000 0010H。

2.3 执行

获得操作数以及操作类型func字段之后，我们输入alu中进行处理，得到相加结果为0000 0011H。

2.4 写回

最后我们再利用访问寄存器堆来写入结果到寄存器rd也就是\$3中。

3. 执行过程总结

- 取值：**从当前 PC 所指地址读取指令。
- 译码：**解析指令字段：op=000000，获取 rs、rt、rd、sa、funct；同时读取寄存器 `rs` 和 `rt` 的值作为操作数。
- 执行：**根据 funct 字段确定操作（如加法、减法等），ALU 计算 `rs` 与 `rt` 的结果。
- 写回：**将 ALU 的计算结果写入 `rd` 寄存器。

(二) I型指令

1. I型指令格式

代码块

1 **31** **26 25** **21 20** **16 15** **0**

2 | op | rs | rt | imm |

- **op**：操作码，决定这是哪种指令（如 lw、sw、addi、beq 等）；
- **rs**：源操作数所在的寄存器编号；
- **rt**：目标寄存器编号；
- **imm**：16 位立即数，可以是地址偏移（如 lw、sw）或是常数（如 addi）。

2. 案例分析 `sw $8, #28($0)`

`sw $8, #28($0)` 这句代码是指，把寄存器\$8中存储的数据，写入#28加上\$0中存储的偏移量的地址的对应内存中去。

2.1 取指

首先是从指令集中取值，我们取出来指令为 `10101100 00001000 00000000 00011100`。

2.2 译码

然后根据对指令译码，我们拆解出op、b、rt、offset这些字段。

并处理为对应信号，以及访问offset、rt、b对应寄存器的数据，得到操作数。

这里我们得到\$8的数据为0000_0011H；\$0的数据为0000_0000H。

2.3 执行

获得地址和偏移量之后之后，我们输入alu中进行相加处理，得到最终存储地址为0000_001CH。

2.4 写回

最后我们再写入结果到计算得到的地址中去也就是0000_001CH中。

3. 执行过程总结

- 取值**：从当前 PC 所指地址读取指令。
- 译码**：获取 op、rs、rt 和 imm 字段；读取 **rs** 的值，并对 imm 进行符号扩展。
- 执行**：根据 funct 字段确定操作（如加法、减法等），ALU 计算 **rs** 与 **imm** 的结果。
- 访存**：**lw**：读取内存地址；**sw**：写数据到内存地址。
- 写回**：**lw**、**addi**：将数据写入 **rt**；**sw**：无写回操作

（三）J型指令

1. J型指令格式

代码块

1	31	26	25	0
2		op		target address

- **op**：操作码，决定是跳转（如j、jal）；
- **target**：目标地址（26位），跳转目标地址的高位由PC当前值决定。

2. 案例分析 j 00H

j型指令是跳转指令，j 00H 这条指令是说下一个指令读取位置跳转到00H这个位置。

2.1 取指

首先是从指令集中取值，我们取出来指令为 00001000 00000000 00000000 00000000。

2.2 译码

然后根据对指令译码，我们拆解出op、target address这些字段。

2.3 写回

我们把target address字段的地址写回next_pc之中。

3. 执行过程总结

- 取值**：从当前PC所指地址获取跳转指令。
- 译码**：提取目标地址字段（26位）。
- 写回**：将提取目标地址写回next_pc之中。

八、多周期CPU原理解释

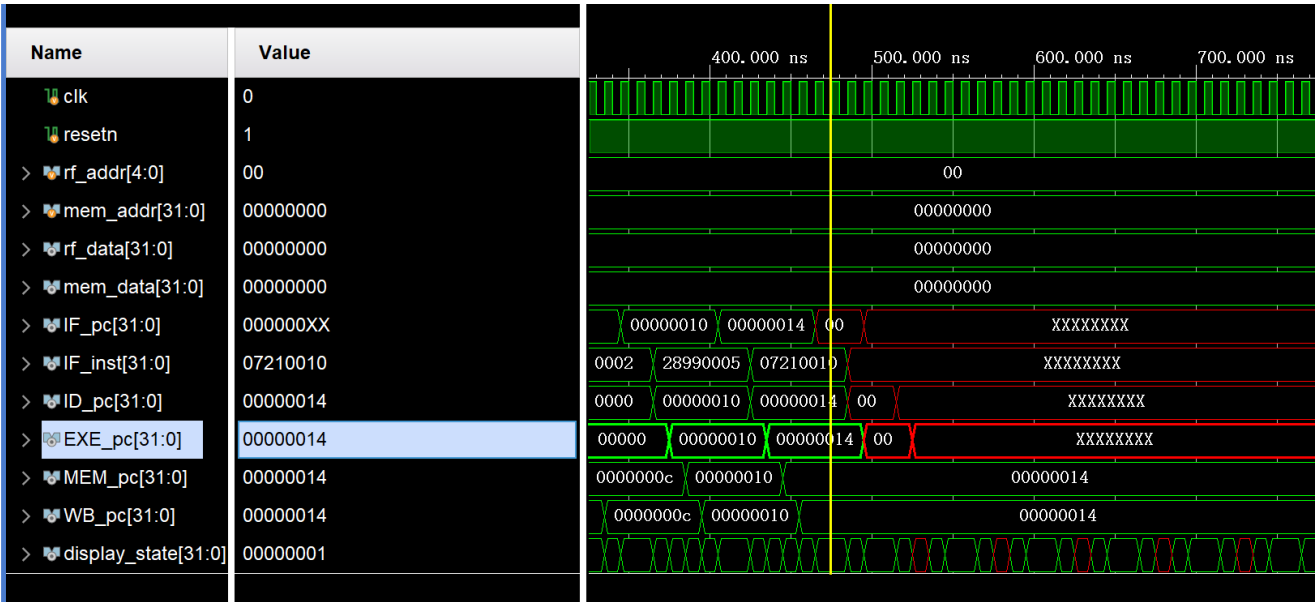
多周期CPU是一种将一条指令的执行过程拆分成多个时钟周期完成的处理器架构，每个周期完成一个子操作（例如取指、译码、执行、访存、写回）。

相比单周期CPU，它能更高效地利用硬件资源，因为每个阶段可以复用电路，而且每条指令所需时间更贴合其实际复杂度。

多周期CPU能够充分发挥流水线的性能，在后面的实验中我们会对流水线进行实验。

九、多周期Bug修复

如果直接按照课本上创建IP核的操作直接运行仿真，我们会发现在第一个跳转指令的地方开始报错。



下面我们来修改ip核的配置选项，来修复bug。

如图片中所示，在**Primitives Output Register**的选项，我们均不打勾。

Component Name data_ram

Basic

Port A Options

Port B Options

Other Options

Summary

Memory Size

Write Width32Range: 8 to 4096 (bits)

Read Width32

Write Depth256Range: 2 to 1048576

Read Depth256

Operating ModeWrite FirstEnable Port TypeAlways Enabled

Port A Optional Output Registers

☐ Primitives Output Register

☐ Core Output Register

☐ SoftECC Input Register

☐ REGCEA Pin

Component Name

Basic

Port A Options

Port B Options

Other Options

Summary

Memory Size

Write Width

32

▼

Read Width

32

▼

Write Depth : 256

Read Depth : 256

Operating Mode

Write First

▼

Enable Port Type

Always Enabled

▼

Port B Optional Output Registers

☐ Primitives Output Register
☐ Core Output Register

☐ SoftECC Output Register
☐ REGCEB Pin

Component Name

Basic

Port A Options

Other Options

Summary

Memory Size

Port A Width

32

✕

Range: 1 to 4608 (bits)

Port A Depth

256

✕

Range: 2 to 1048576

The Width and Depth values are used for Read Operation in Port A

Operating Mode

Write First

▼

Enable Port Type

Always Enabled

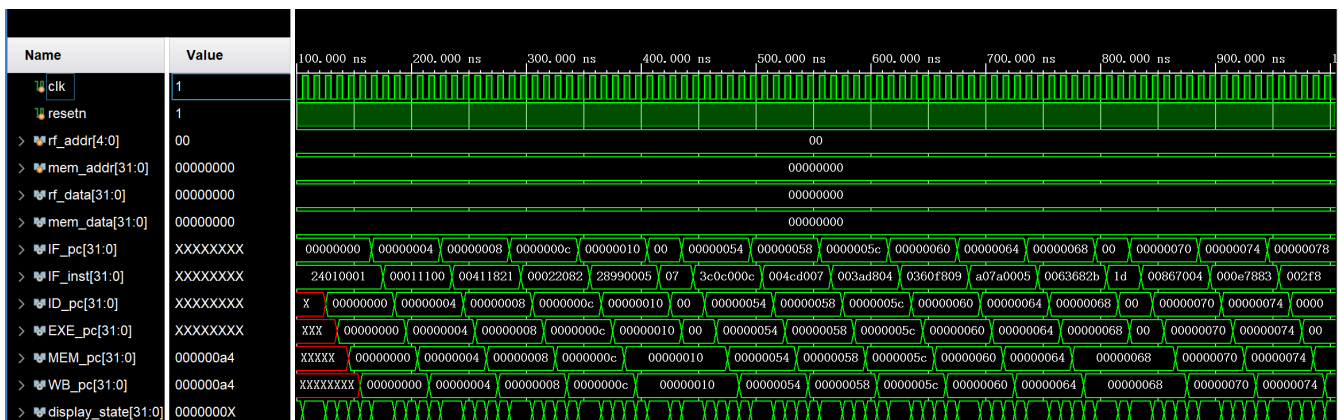
▼

Port A Optional Output Registers

☐ Primitives Output Register
☐ Core Output Register

☐ SoftECC Input Register
☐ REGCEA Pin

修正之后就能正常运行多周期cpu了！



十、多周期CPU功能添加

1. 修改multi_cycle_cpu

要修改多周期中ID_EXE的总线长度。

代码块

```
1  wire [152:0] ID_EXE_bus; // ID->EXE级总线
2
3  reg [152:0] ID_EXE_bus_r;
```

2. 修改ID_model

在指令集中取出指令之后，我们要对pc指令译码，所以我们先来修改译码部分。直接仿照SLT指令和XOR指令添加即可，下面为添加的片段：

代码块

```
1  // 1.修改ID->EXE总线长度，因为独热编码变长了三个单位
2  output [152:0] ID_EXE_bus,
3  ...
4  //2.添加实现指令列表，这里需要注意还要添加立即数相关的I型指令，以及对应的编码不能重复
5  wire inst_BIG, inst_BIGU, inst_BIGI, inst_BIGIU, inst_AND_NOT,
   inst_AND_NOTI;
6  ...
7  assign inst_BIG = op_zero & sa_zero & (funct == 6'b111010); //大于则置位,
   自定义
8  assign inst_BIGU = op_zero & sa_zero & (funct == 6'b111011); //无符号大则
   置, 自定义
9  assign inst_BIGI = (op == 6'b011000); // 自定义: 有符号大于立即数
10 assign inst_BIGIU = (op == 6'b011001); // 自定义: 无符号大于立即数
11 assign inst_AND_NOT = op_zero & sa_zero & (funct == 6'b111100); //yu非运
   算, 自定义
12 assign inst_AND_NOTI = (op == 6'b111101);
13 ...
14 //3.添加alu操作分类
15 wire inst_big, inst_bigu, inst_and_not;
16 assign inst_big = inst_BIG | inst_BIGI;
17 assign inst_bigu = inst_BIGIU | inst_BIGU;
18 assign inst_and_not = inst_AND_NOT | inst_AND_NOTI;
19 ...
20 //4.添加到依据立即数扩展方式分类
```

```

21  assign inst_imm_zero = inst_ANDI | inst_LUI | inst_ORI | inst_XORI |
    inst_AND_NOTI;
22  assign inst_imm_sign = inst_ADDIU | inst_SLTI | inst_SLTIU | inst_BIGI |
    inst_BIGIU
23                          | inst_load | inst_store;
24  //添加到依据目的寄存器号分类
25  assign inst_wdest_rt = inst_imm_zero | inst_ADDIU | inst_SLTI
26                          | inst_SLTIU | inst_load | inst_BIGI | inst_BIGIU;
27  assign inst_wdest_rd = inst_ADDU | inst_SUBU | inst_SLT | inst_SLTU
28                          | inst_JALR | inst_AND | inst_NOR | inst_OR
29                          | inst_XOR | inst_SLL | inst_SLLV | inst_SRA
30                          | inst_SRAV | inst_SRL | inst_SRLV | inst_BIG
31                          | inst_BIGU | inst_AND_NOT;
32  ...
33  //5.添加独热编码
34  assign alu_control = {
35                          inst_big,
36                          inst_bigu,
37                          inst_and_not,
38                          inst_add,           // ALU操作码, 独热编码
39                          inst_sub,
40                          inst_slt,
41                          inst_sltu,
42                          inst_and,
43                          inst_nor,
44                          inst_or,
45                          inst_xor,
46                          inst_sll,
47                          inst_srl,
48                          inst_sra,
49                          inst_lui
50  };

```

3. 修改EXE_module

修改完译码之后，我们来修改执行部分。

代码块

```

1  input    [152:0] ID_EXE_bus_r, // ID->EXE总线
2  wire     [14:0] alu_control;

```

除了修改输入和变量之外，这部分主要要修改alu，修改方式和之前的实验一致。这里就不贴代码了。

4. 修改coe文件

我们在coe文件里面加入这三行，分别代表有符号数大于置位、无符号数大于置位、与非操作。

代码块

```
1 00000000010000010001100000111010 //BIG $2 $1 $3
2 00000000010000010001100000111011 //BIGU $2 $1 $3
3 00000000011001100100000000111100 //NAND $3 $6 $8
4 0041183A
5 0041183B
6 0066403C
```

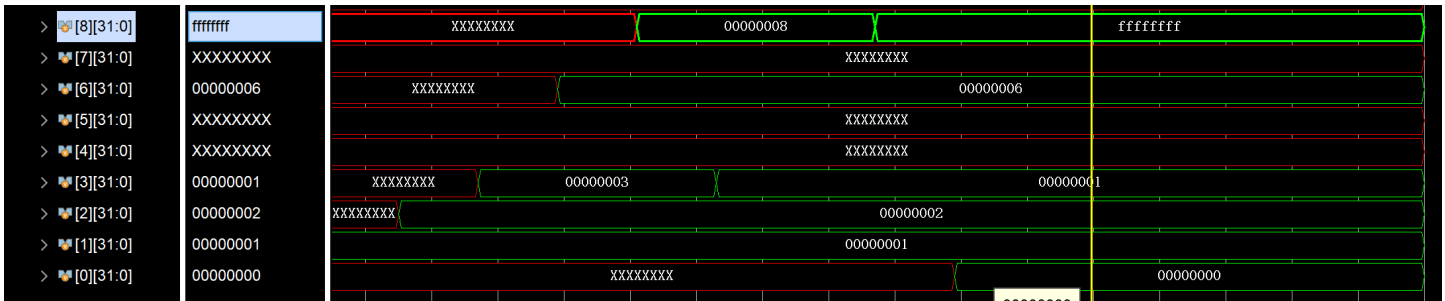
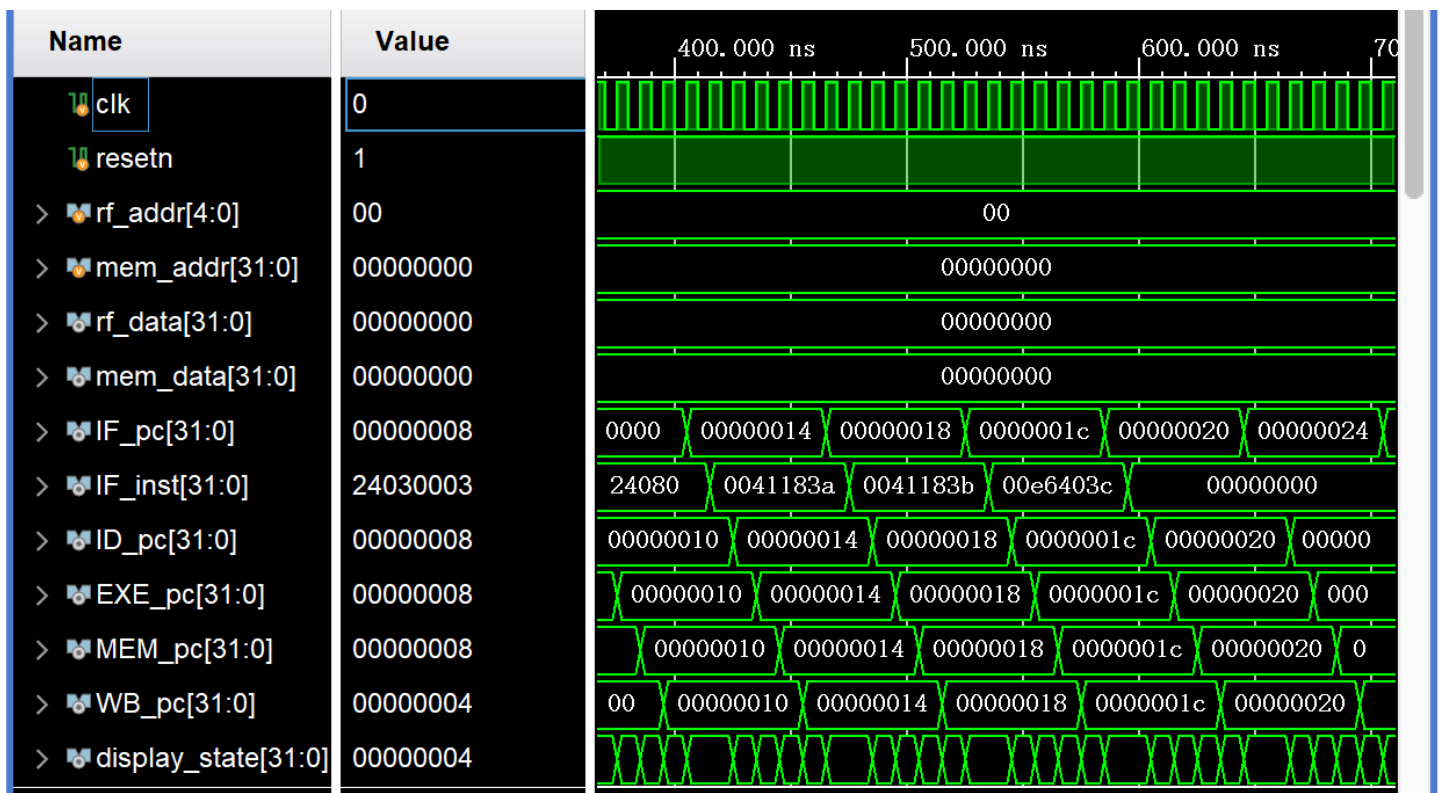
五、实验结果分析

我们重新设计了coe文件，分别对\$1\$2\$3\$6\$8赋值为1、2、3、6、8，然后去验证我们加的功能指令先是对\$2和\$1分别做大于置位，赋值到\$3上，使得\$3为1，先下图结果可知正确。

然后是对\$3和\$6做一个与非，得到ffffffff赋值到\$8上，从下图可知结果正确。

代码块

```
1 24010001
2 24020002
3 24030003
4 24060006
5 24080008
6 0041183A
7 0041183B
8 0066403C
```



六、总结心得

- 本次实验真的做了好久好久，浪费了很多时间，幸好老师延期了，在这给董老师跪一个
- 本次实验真真切切的学会了如何debug，才知道可以直接拖变量到simulation里面进行查看。一通debug下来，结果发现是自己添加功能的时候一堆写错太粗心了，导致一直调不出来。
- 本次实验做了很久，现在对cpu的流程真的倒背如流了，超级熟悉了单周期cpu和多周期cpu的工作原理。也对各种指令是如何译码解析，然后执行的有了更加深刻的理解。
- 最后还是感谢董老师带我debug，给董老师点赞。